

Reference

Acetilsalicylic

April 8, 2026

Contents

1 Template

- 1.1 Acetil's template
- 1.2 Alternative to bits/stdc++.h

2 Language stuff

- 2.1 Primitive types
- 2.2 Useful macros
 - 2.2.1 Limits

3 Algorithms

- 3.1 Binary search
 - 3.1.1 Fast square root
- 3.2 Misc
 - 3.2.1 Binary exponentiation
 - 3.2.2 Mid point calculation
- 3.3 Segment Tree
 - 3.3.1 Recursive
 - 3.3.2 Iterative
 - 3.3.3 Lazy Propagation

4 Data Structures

- 4.1 Ordered Set

5 Maths

- 5.1 Value of pi
- 5.2 *Sigma function (σ)

1 Template

1.1 Acetil's template

Listing 1: Acetil's template

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 /* num types */
4 typedef long long ll;
5 typedef unsigned long long ull;
6
7 /* data structures */
8 #define mapi map<int, int>
9 #define veci vector<int>
10 #define seti set<int>
11 #define pai pair<int, int>
12 #define F first
13 #define S second
14
15 /* algos */
16 #define fori(n) for (int i=0; i<n; i++)
17 #define fort(t) while(t--)
18 #define fork(k, n) for (int i=k; i<n; i++)
19
20 /* fast IO */
21 #define fastio \
22     ios::sync_with_stdio(0); cin.tie(0)
23
24 /* constants */
25 constexpr int imax = INT_MAX;
26 constexpr ll llmax = LLONG_MAX;
27 constexpr int imin = INT_MIN;
28
29
```

```
30
31 /* solution */
32 void solve() {
33
34 }
35 int main() {
36
37 }
```

1.2 Alternative to bits/stdc++.h

Sometimes stdc++ can't be imported. Instead

Listing 2: Alternative to bits/stdc++.h

```
1 #include <iostream>
2 /* for sort and stuff */
3 #include <algorithm>
4 /* data structures */
5 #include <set>
6 #include <vector>
7 #include <map>
```

2 Langue stuff

2.1 Primitive types

keyword	Bytes	M ₂	m ₂	M ₁₀	m ₁₀
int	4	2 · 10 ⁹	−2 · 10 ⁹	2 ³¹ − 1	−2 ³¹
long long	8	2 ⁶³ − 1	−2 ⁶³	9 · 10 ¹⁸	−9 · 10 ¹⁸

2.2 Useful macros

2.2.1 Limits

- INT_MAX: Max integer value
- INT_MIN: Min integer value
- LONG_MAX: Max long int value
- LONG_MIN: Min long int value
- LLONG_MAX: Max long long int value
- LLONG_MIN: Min long long int value

3 Algorithms

3.1 Binary search

3.1.1 Fast square root

3.2 Misc

3.2.1 Binary exponentiation

Calculate expontents in logarithmic time.

Listing 3: Binary exponentiation

```
1 long long binpow(long long a, long long n) {
2     long long ans = 1;
3     while (n > 0) {
4         if (n & 1)
5             ans *= a;
6         a *= a;
7         n >>= 1;
8     }
9     return ans;
10 }
```

Modulo Somentimes the power is just too high, so if doing modulo is allowed, following the property $(a \times b)(mod\ m) = [(a\ mod\ m) \times (b\ mod\ m)](mod\ m)$:

Listing 4: Binary exponentiation

```
1 long long binpow(long long a, long long n, long long m) {
2     long long ans = 1;
3     while (n > 0) {
4         if (n & 1)
5             ans = ans * a % m;
6         a = a * a % m;
7         n >>= 1;
8     }
9     return ans;
10 }
```

3.2.2 Mid point calculation

3.3 Segment Tree

A segment tree is stored as a vector of the datatype it stores. For simple problems, an int type is enough. For more complex or lazy propagation, a Node structure is required.

The segtree is stored in a list, that must have a size of $4n$, where n is the size of the given array a . This is necessary so the binary tree that holds the segtree is complete and can behave correctly.

Elements of a SegTree To build a segment tree, these things are required: an operation, that must be **associative**, have an **identity element** and, preferably, it must be **commutative**.

Structure Each node of a segtree has a node, and given a node i , it's left child is at index $2i$, and it's right children is at $2i + 1$. The root is at $i = 1$.

Operations The operations on a SegTree usually include the following:

- The index of the node, usually i
- The range over the list, $[l, r]$, where $l \geq 0$; $r < n$.

It's important that the range over the list is used as **zero indexed**, because the original array is zero indexed too.

3.3.1 Recursive

Intuition Each recursive function call represent a current node. This is achieved by passing an i parameter, which is the index of the node inside of the vector representing the tree. Also, two other parameters (tl and tr, standing for temporal low and temporal high respectively) represent the “reach”, or the interval over the original list of elements a that the current node represents (as $[tl, th]$).

Elements The following are the elements required for a recursive segment tree. Each element is defined as following:

- vector<int> tree: The tree itself
- vector<int> a: The original list of elements, read directly from stdin.
- int n: The size of a, the number of elements on the list input.
- int N: The length of the vector that holds the tree, $N = 4n$

Listing 5: Recursive segtree declaration stuff - common.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> tree; // size() = len = N * 2
5 vector<int> a; // size() = n
6 int n, N;
```

Building is done starting from the root, where $i = 1$. The recursive function descends through every branch, reducing the segment that each node covers (where the interval covered by node i is defined as $[tl, tr]$) in half each time, similar to a binary search.

Listing 6: Recursive build function

```
1 #include "common.cpp"
2
3 /* build the tree */
4 void build(int tl=0, int tr=n-1, int i=1) {
5     // range is [tl, tr]. If tr < tl, it isn't a valid range
6     if (tr < tl) return;
7     // if tl == tr, we've reached a leaf
8     if (tl == tr) {
9         // set the leaves of the tree
10        tree[i] = a[tl];
11        return;
12    }
13
14    int mid = tl + (tr - tl)/2;
15
16    // update the children recursively
17    build(tl, mid, i<<1);
18    build(mid+1, tr, i<<1^1);
19
20    // update parent with new children's values
21    // desired operator instead '+'
22    tree[i] = tree[i<<1] + tree[i<<1^1];
23 }
```

Querying is done combining the top-most nodes that are inside of the range.

Listing 7: Recursive query function

```
1 #include "common.cpp"
2
3 /* query the tree */
4 int query(int l, int r, int tl=0, int tr=n-1, int i=1) {
5     if (tr < l || tl > r)
6         return 0; // Identity element instead of '0'
7     if (tl >= l && tr <= r)
8         // If current node is in range, then return it's value
9         return tree[i];
10
11    int mid = tl + (tr-tl)/2;
12
13    // Return the combination of the two halves. This is why
```

```

14 // an identity element is nice: if some half is not valid,
15 // then it won't affect the correct result.
16 //
17 // desired operation instead of '+'
18 return query(l, r, tl, mid, i<<1)
19     + query(l, r, mid+1, tr, i<<1^1);
20 }

```

Updating a position implies doing some binary-search-like traversal and updating a leaf and its parents.

Listing 8: Recursive update function

```

1 #include "common.cpp"
2
3 /* update a position */
4 void update(int pos, int val, int tl=0, int tr=n-1, int i=1) {
5     if (tr < tl) return;
6     if (tr < pos || tl > pos) return;
7     if (tl == tr) {
8         if (tl != pos) return;
9         tree[i] = val;
10        return;
11    }
12
13    int mid = tl + (tr - tl)/2;
14
15    update(pos, val, tl, mid, i<<1);
16    update(pos, val, mid+1, tr, i<<1^1);
17
18    // Desired operation instead of '+'
19    tree[i] = tree[i<<1] + tree[i<<1^1];
20 }

```

3.3.2 Iterative

The iterative version walks the tree starting from the leaves, and walks up until it reaches the root. The representation on the vector is the same.

Building It's not necessary to have a special build function. Utilizing the update function can accomplish the same goal.

Update Now, the representation in memory is as follows: The original array's elements are on the second half of the vector. So if we have n elements in array a , then the first element a_1 will be on position $tree[2n]$, because we keep the array 0-indexed. Then

the rest of the tree is found on the positions $i \in [1, 2n - 1]$. So, to update, pos must be the 0-indexed position of the element that will be updated. (TODO: is this true?)

Listing 9: Iterative segtree elements

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> tree;
5 vector<int> a;
6
7 int n, N;

```

Listing 10: Iterative update/build function

```

1 #include "common.cpp"
2
3 /* iterative update */
4 void update(int pos, int val) {
5     tree[pos+=n<<1] = val;
6     for (pos>>=1; pos >=1; pos>>=1)
7         /* merge nodes operator instead of '+' */
8         tree[pos] = tree[pos<<1] + tree[pos<<1^1];
9 }

```

Query Querying the tree is different from the recursive approach, cause we start on the leaves. So if the query interval is $[a, b]$, then we will work with $[a, b + 1]$. Now, a and b must be 0-indexed.

Listing 11: Iterative query

```

1 #include "common.cpp"
2
3 /* l and r are zero indexed */
4 int query(int l, int r) {
5     int ans = 1; /* identity element */
6     for (l += n<<1, r += (n<<1) + 1; l < r; l>>=1, r>>=1) {
7         /* merge nodes operator instead of '+=' */
8         if (l & 1)
9             ans += tree[l++];
10        if (r & 1)
11            ans += tree[--r];
12    }
13    return ans;
14 }

```

3.3.3 Lazy Propagation

Lazy propagation solves the problem for update queries on an interval, they can be set queries, sum queries, or anything involving modifying an interval. The idea is queue the modification on a parent node, until it becomes necessary to update any further. It's more easily implemented in a recursive way.

Example This example segment tree is prepared for the following queries:

- Range sum query with a, b, k
- Range set query with a, b, k
- Range sum query with a, b

Listing 12: SegTree & Lazy elements

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     // All default values so they create
6     // the identity element. Initial value
7     // should represent the identity, as well
8     // as the other lazy fields
9     int value = 0;
10
11     bool lazy_sum_flag = false;
12     bool lazy_set_flag = false;
13
14     int lazy_sum_value = 0;
15     int lazy_set_value = 0;
16
17     // "Combine" operator. Can be defined
18     // as needed by the problem.
19     Node operator+ (const Node& a) {
20         // We build a new instance that combines
21         // two nodes
22         return {
23             .value = this->value + a.value
24         };
25     }
26 };
27
28 // This function is the heart of lazy prop
29 // Allows to flush or push the delayed updates
30 // stored in the parent to the children, so
```

```
31 // we're one step closer to updating the
32 // leaf nodes
33 void push(
34     Node& parent,
35     Node& left_child,
36     int left_tl,
37     int left_tr,
38     Node& right_child,
39     int right_tl,
40     int right_tr
41 ) {
42     // If parent doesn't have pending updates,
43     // then we do nothing
44     if (parent.lazy_sum_flag) {
45         left_child.lazy_sum_flag = true;
46         // This operation must combine with any
47         // possible previously added values. If
48         // it overrides it, then we lose it. As this
49         // is a sum, it must be summed
50         left_child.lazy_sum_value +=
51             parent.lazy_sum_value;
52         // We must also update the values of the
53         // children, not only add to their
54         // delayed operations
55         left_child.value +=
56             (left_tr - left_tl + 1) * parent.lazy_sum_value;
57
58         right_child.lazy_sum_flag = true;
59         right_child.lazy_sum_value +=
60             parent.lazy_sum_value;
61         right_child.value +=
62             (right_tr - left_tl + 1) * parent.lazy_sum_value;
63
64         // As we've pushed the pending update, the
65         // parent doesn't need to store it as
66         // pending. It already passed it to it's
67         // children.
68         parent.lazy_sum_flag = false;
69         parent.lazy_sum_value = 0;
70     }
71
72     if (parent.lazy_set_flag) {
73         left_child.lazy_set_flag = true;
74         // This overrides, cause it's the nature
75         // of the type of query
```

```

76     left_child.lazy_set_value =
77         parent.lazy_set_value;
78     left_child.value =
79         (left_tr - left_tl + 1) * parent.lazy_set_value;
80
81     right_child.lazy_set_flag = true;
82     right_child.lazy_set_value =
83         parent.lazy_set_value;
84     right_child.value =
85         (right_tr - right_tl + 1) * parent.lazy_set_value;
86
87     parent.lazy_set_flag = false;
88     parent.lazy_set_value = 0;
89 }
90 }
91
92 int n, N;
93 vector<Node> tree;
94 vector<int> a;

```

Build While building, is important to correctly represent the identity. If we try to combine two nodes a and b with the custom $+$ operator, and imagine a is initialized but b isn't, then we must ensure that $a + b = a$ using the identity.

Listing 13: SegTree build function

```

1 #include "common.cpp"
2
3 void build(int tl=0, int tr=n-1, int i=1) {
4     // the range of the current node is [tl, tr]
5     // if tr < tl, then it's not a valid range
6     if (tr < tl) return;
7
8     // We've reached a leaf
9     if (tl == tr) {
10         // Since i is the node's index on the tree,
11         // and tl = tr represents the range over the original
12         // array a, then we need to initialize this leaf
13         // to it's corresponding value on the array
14         tree[i].value = a[tl];
15         return;
16     }
17
18     int mid = tl + (tr - tl)/2;
19

```

```

20     build(tl, mid, i<<1);
21     build(mid+1, tr, i<<1^1);
22
23     // We use the custom combine "+" operator here
24     // to build the parent from the already initialized
25     // children.
26     tree[i] = tree[i<<1] + tree[i<<1^1];
27 }

```

4 Data Structures

4.1 Ordered Set

Exclusive of the GCC compiler collection.

Listing 14: PBDS ordered set declaration

```

1 // Common file
2 #include <ext/pb_ds/assoc_container.hpp>
3 // Including tree_order_statistics_node_update
4 #include <ext/pb_ds/tree_policy.hpp>
5
6 using namespace std;
7 using namespace __gnu_pbds;
8
9 typedef tree<
10     int,
11     null_type,
12     less<int>,
13     rb_tree_tag,
14     tree_order_statistics_node_update
15 > ordered_set;

```

5 Maths

5.1 Value of pi

The value of π is not always available. It can be obtained as $\cos^{-1}(-1)$.

5.2 *Sigma function (σ)